



Pontificia Universidad Católica de Chile
Escuela de ingeniería
Departamento de ciencias de la
computación Profesor: Hernán Cabrera

Pauta Interrogación 2

IIC2513 Tecnologías y aplicaciones WEB

10 de junio 2025 17:30 hrs

1. RFC5789 define PATCH., explique: ¿cuáles son las diferencias entre PATCH y PUT? ¿Qué es un RFC? (explique con sus propias palabras)

PATCH se utiliza para realizar actualizaciones parciales sobre un recurso. En cambio, PUT puede utilizarse para crear o reemplazar completamente un recurso en una URI específica. Aunque PUT también puede realizar actualizaciones, estas implican sobrescribir completamente el recurso existente.

Lo importante es que PATCH es para update, eso es lo central. Si bien PUT se puede usar para updates, esto es reemplazando completamente el recurso y no una parte del mismo, **(1,5 puntos)**

Un RFC (Request For Comments) es un documento técnico que contiene especificaciones y estándares formales para protocolos y tecnologías utilizadas en internet.

RFC es el request for comments, lo central es que es el lugar para encontrar información formal para protocolos. **(0,5 puntos)**

2. Según RFC2616 y RFC5789, comente/explique/de un ejemplo el siguiente párrafo : "PUT pone (crea) un recurso en una URI particular sólo si no hay ningún recurso en dicha URI. Si ya existe algún recurso en esa URI, PUT no cambia ese recurso (Para eso está PATCH)."

PUT se utiliza para crear o reemplazar totalmente un recurso en una URI. Si ya existe un recurso en dicha URI, PUT no lo modifica parcialmente. Para realizar cambios parciales se utiliza PATCH.

La clave es entender la función de PUT como "creación/actualización total" en este caso ejemplos pueden ser cuando llega información nueva sobre un recurso y lo tenemos que actualizar completamente con otro registro que lo reemplace. por ejemplo el contenido de una casilla determinada de un juego de ajedrez. **(2 puntos por una explicación coherente del método, 1 punto por explicación buena pero parcial)**

3. El siguiente trozo de código se caerá, ¿Por qué? y ¿por qué no se cae si se comenta la línea que dice "let variable = 505"?

El error ocurre porque se intenta declarar la variable 'variable' dos veces



Pontificia Universidad Católica de Chile
Escuela de ingeniería
Departamento de ciencias de la
computación Profesor: Hernán Cabrera

utilizando 'let', lo cual no está permitido. Si se comenta la segunda declaración 'let variable = 505', no hay redeclaración y el código funciona correctamente.

No se puede volver a declarar una variable con let (2 puntos)

4. Comente por qué la salida de las líneas siguientes será 505 y, luego el programa no se caerá.

Con 'var' sí se permite redeclarar una variable. Por eso, al ejecutar el código, la última asignación 'var variable2 = 505' sobrescribe la anterior, y el valor mostrado por consola es 505.

Porque se usa var, en ese caso se puede reescribir/redefinir. (2 puntos)

5. Aquí tiene un ejemplo de test utilizando la librería jest. Test con Jest sobre createPlayer (4 puntos)

¿Qué función se está testeando? La función que se está testeando es 'createPlayer'. (1 punto)

¿Qué se espera que haga esa función? Se espera que cree un jugador con los datos entregados (username y email) y retorne un estado HTTP 201. (1 punto)

¿Qué valores se simulan y se esperan? El valor simulado es la respuesta de 'Player.create', que retorna un objeto mock con id, username, email y score. El test verifica que esta función se haya llamado correctamente y que el estado y el cuerpo de la respuesta coincidan con los esperados. (2 puntos)

6. En una promesa, ¿cuáles son los estados que puede tener?, explique cada uno de ellos.

1. Pending (Pendiente)

Es el estado inicial de una promesa. Significa que la operación asíncrona aún no ha terminado, por lo tanto, ni se ha resuelto ni se ha rechazado. Ejemplo: se está esperando la respuesta de una solicitud a una API.

2. Fulfilled (Cumplida / Resuelta)

La operación se completó correctamente. La promesa se resuelve y produce un valor que se puede manejar usando .then().

```
fetch('/api/datos')
```

```
.then(response => { // Aquí la promesa está fulfilled });
```

3. Rejected (Rechazada): La operación falló por alguna razón (por ejemplo, un error de red). La promesa se rechaza y entrega una razón o error, que se puede manejar con .catch().



Pontificia Universidad Católica de Chile
Escuela de ingeniería
Departamento de ciencias de la
computación Profesor: Hernán Cabrera

```
fetch('/api/datos')  
.catch(error => { // Aquí la promesa está rejected });
```

Pending (Pendiente), fulfilled (Completada), rejected (rechazada)
(0,5 puntos por nombrar los 3 correctamente. Se puede asignar puntaje parcial proporcional por algún estados correcto 0,15 para 1 o 0.25 si tiene 2 correctos)

La explicación de cada uno 0,5. Debe estar asociada correctamente al nombre
(total 1,5 puntos)

7. ¿Por qué necesitamos promesas en JavaScript? ¿Qué se quiere lograr?

Necesitamos promesas en JavaScript porque nos permiten manejar operaciones asíncronas (como solicitudes a servidores, lectura de archivos, temporizadores, etc.) de una forma más clara, estructurada y manejable. Buscar resolver algunos problemas que surgen con la asincronía como: evitar el callback hell, manejar errores de forma centralizada y encadenar operaciones de forma ordenada.

Explicación debe estar en torno a la asincronía y resolver los problemas que esto pudiese traer. **(2 puntos)**

8. El Browser es un software complejo que permite, entre otras cosas, renderizar lo que se envía desde un servidor (o archivos locales también). ¿Cuáles son las dos estructuras que arma? (HINT DOM) Con esas dos estructuras ¿Qué construye?

El navegador (browser) construye dos estructuras principales a partir de los archivos que recibe (HTML y CSS):

1. HTML DOM (Document Object Model): Es una representación estructurada del contenido HTML del documento. Cada etiqueta HTML se convierte en un nodo dentro de un árbol jerárquico que refleja la estructura del documento.

2. CSSOM (CSS Object Model): Es una representación del estilo aplicado al documento. El navegador analiza las reglas CSS (ya sea en archivos externos, `<style>`, o `style` en línea) y construye una estructura que indica qué estilos se aplican a qué elementos.

¿Qué se construye con ambas? Con el DOM y el CSSOM, el navegador construye el Render Tree

Es una estructura que contiene los elementos visuales que realmente se van a mostrar en la pantalla, combinando el contenido (DOM) y los estilos



Pontificia Universidad Católica de Chile
Escuela de ingeniería
Departamento de ciencias de la
computación Profesor: Hernán Cabrera

(CSSOM).

El render tree permite al navegador saber qué dibujar, cómo y dónde, y es la base para los siguientes pasos del proceso de renderizado: layout (calcular posiciones) y painting (dibujar en pantalla).

1 punto: CSSOM y HTML DOM **y 1 punto** por el render tree

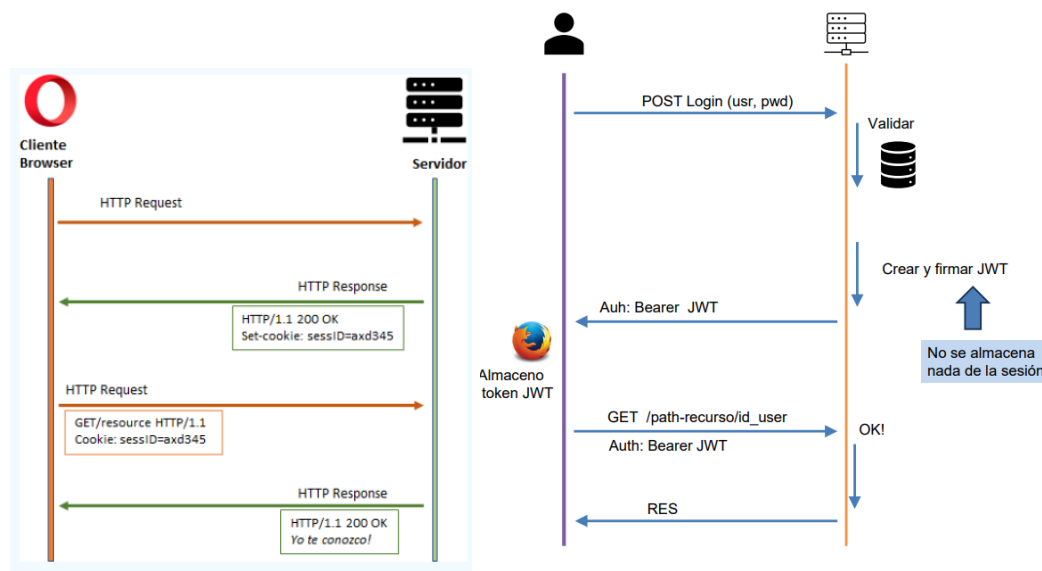
9. ¿Para qué sirve el archivo .env? ¿Cuál es el objeto JavaScript que permite acceder a la información en .env?

El archivo .env se usa para almacenar variables de entorno, como claves secretas, credenciales o configuraciones específicas (puerto del servidor, URL de una base de datos, etc.). Se utiliza principalmente para proteger información sensible y separar la configuración del código fuente, lo que mejora la seguridad y privacidad del proyecto.

El objeto que permite acceder a la información en .env es: process.env . Este objeto está disponible en entornos como Node.js y permite acceder a las variables definidas en el archivo .env.

1 punto por explicación con privacidad y/o seguridad. **1 punto** por hablar del objeto de JS process

10. Dibuje el diagrama de secuencia entre el cliente y el servidor para las cookies y para JWT (4 puntos)



2 puntos por cada diagrama (están en ppt de las clases)



Pontificia Universidad Católica de Chile
Escuela de ingeniería
Departamento de ciencias de la
computación Profesor: Hernán Cabrera

11. ¿Cuáles son los 3 componentes de un JWT?

1. Header (Encabezado): Contiene información sobre el tipo de token (JWT) y el algoritmo de firma usado (por ejemplo, HS256).

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

2. Payload (Cuerpo o carga útil): Contiene los datos (claims) del usuario u otra información.

Estos datos no están encriptados, solo codificados en base64.

```
{  
  "userId": 123,  
  "role": "admin"  
}
```

3. Signature (Firma): Es el resultado de aplicar un algoritmo de firma (como HMAC o RSA) al header y el payload usando una clave secreta.

Sirve para verificar que el token no ha sido alterado.

```
HMACSHA256(  
  base64UrlEncode(header) + "." + base64UrlEncode(payload),  
  secret  
)
```

Mencionar sus componentes: Header, payload, signature. explicar brevemente (2 pts)

12. ¿Por qué no es bueno colocar una password dentro de un JWT?

Porque un JWT no garantiza confidencialidad, solo garantiza integridad y autenticidad.

Los datos del payload no están cifrados, solo están codificados en base64, por lo tanto, cualquiera que tenga el token puede leer su contenido fácilmente.

Aunque el token esté firmado (signature), eso solo asegura que no ha sido modificado, pero no oculta la información sensible.

Por eso, nunca debe incluirse una contraseña o datos privados dentro del payload de un JWT. Si se necesita confidencialidad, se debe usar cifrado adicional.



Pontificia Universidad Católica de Chile
Escuela de ingeniería
Departamento de ciencias de la
computación Profesor: Hernán Cabrera

Explicar que el JWT no asegura confidencialidad. (2 pts)

13. ¿Por qué está mal este trozo de código? Asumamos que existe la función `soyAsync()` que devuelve de manera asíncrona un valor.

El código está mal porque no se puede usar `await` fuera de una función `async`.

En JavaScript, el operador `await` solo puede ser utilizado dentro de funciones declaradas con `async`.

`await` pausa la ejecución de la función hasta que la promesa se resuelva, y eso solo tiene sentido dentro de una función `async`, que devuelve implícitamente una promesa. Usar `await` sin `async` produce un `SyntaxError` en tiempo de ejecución.

1 punto: `await` solo puede usarse dentro de una función declarada como `async`.

1 punto: El código está mal porque `unaFuncion()` no es `async`, entonces no puede usar `await` dentro de ella.

`await` sin `async` no se puede (2pts)

14. ¿Cuál es la razón por la cuál en React se implementaron los hooks? Responda desde el punto de vista de los componentes de React. Su arquitectura (DOM) y su relación con el DOM del browser. Lo vimos en clases *.(3 puntos)

Los hooks se implementaron para manejar el estado y el ciclo de vida de los componentes funcionales, que originalmente no tenían esa capacidad.

React no actualiza directamente el DOM del navegador cuando cambia el estado. En cambio, mantiene una estructura interna llamada Virtual DOM, que es una representación en memoria del árbol de componentes. Cuando se usa un hook como `useState` o `useEffect`, se crea un espacio de memoria especial que React puede asociar a cada componente funcional, para recordar valores (estado) o ejecutar efectos (como renderizar de nuevo, sincronizar datos, etc.).

Los componentes funcionales originalmente eran estáticos y no podían tener estado ni responder al ciclo de vida. Para eso se usaban componentes de clase, que eran más complejos.

Con hooks, se puede escribir componentes más simples y reutilizables sin clases, pero con toda la capacidad de reacción al estado y al DOM.

La estructura virtual de react no se vincula directamente con los cambios del



Pontificia Universidad Católica de Chile
Escuela de ingeniería
Departamento de ciencias de la
computación Profesor: Hernán Cabrera

estado en el DOM del browser por lo tanto se crea un espacio de memoria (una estructura) que los vincula a ambos (el hook) Respuesta completa: **(3 puntos. Parcial: 1,5 punto)**

La respuesta entregada en esta pauta es referencial, el alumno puede responder con sus palabras pero que quede claro que los cambios de estado no se reflejan en el render del componente React de manera “nativa” sino que hay que implementar estructuras adicionales.

15. ¿Esta línea es de un JSX o de HTML? Justifique su respuesta.

`<p className——“id”>Esto es un párrafo</pa`

En HTML se usa el atributo class, pero en JSX (que se usa en React), el atributo class está reservado como palabra clave en JavaScript. En la línea dada se usa className, lo que solo es válido en JSX, no en HTML tradicional.

Es un JSX. En HTML es class, no className (2pts)

16. Para el siguiente trozo de código. ¿Por qué no funciona el log (logger en este caso) si el resto está funcionando todo bien? Explique.

El problema es que el middleware logger no se está usando, por eso no funciona el log aunque el resto del servidor esté bien configurado. Falta la línea `app.use(logger());` `app.use()` se utiliza en Koa para registrar middlewares, que son funciones que se ejecutan antes o después de que se procese una solicitud.

No se hizo el “use”. Explicar para qué sirve esto (2pts)

17. ¿Qué característica de la tecnología WEB (que sabemos que cumple con ser REST) hizo necesario el implementar cookies? Explique y dé un ejemplo.

El protocolo HTTP y los servicios REST son stateless, es decir, no mantienen estado entre solicitudes. Cada petición que se hace al servidor es independiente y el servidor no recuerda información previa del cliente. Para mantener la sesión o la identidad del usuario a través de múltiples peticiones, ya que sin estado, el servidor no sabe quién está haciendo la solicitud ni si ya inició sesión.

Ejemplo: Cuando un usuario inicia sesión en un sitio web, el servidor responde con una cookie de sesión que el navegador almacena. En las siguientes solicitudes, el navegador envía esta cookie para que el servidor reconozca al



Pontificia Universidad Católica de Chile
Escuela de ingeniería
Departamento de ciencias de la
computación Profesor: Hernán Cabrera

usuario y mantenga su sesión activa.

Explicación de stateless: 1 punto

Ejemplo: 1 punto

18. Explique paso a paso que hace este código y su resultado final. (puede referenciar, en su respuesta, a las líneas por su letra, por ejemplo, la línea c) realiza.)
HINT: en el HTML hay una lista ordenada (ol) llamada "lista". como visto en clases! (3 puntos)

```
a. let lista= document.getElementById("lista");  
b. let nuevoLi = document.createElement("li");  
c. nuevoLi.id = "nuevo";  
d. let nuevoTexto = document.createTextNode("HOLA MUNDO");  
e. nuevoLi.appendChild(nuevoTexto);  
f. lista.appendChild(nuevoLi);
```

a) `let lista = document.getElementById("lista");`
Busca en el documento el elemento con id "lista" (una lista ordenada `<ol>`) y lo guarda en la variable `lista`.

Nota: debería ser `getElementById` (la E mayúscula y l minúscula).

b) `let nuevoLi = document.createElement("li");`
Crea un nuevo elemento `<li>` (un ítem de lista) y lo guarda en `nuevoLi`.

c) `nuevoLi.id = "nuevo";`
Asigna el atributo `id="nuevo"` al nuevo `<li>` creado.

d) `let nuevoTexto = document.createTextNode("HOLA MUNDO");`
Crea un nodo de texto con el contenido "HOLA MUNDO" y lo guarda en `nuevoTexto`.

e) `nuevoLi.appendChild(nuevoTexto);`
Añade el nodo de texto como hijo del nuevo `<li>`.

f) `lista.appendChild(nuevoLi);`
Añade el nuevo `<li>` con el texto dentro de la lista ordenada `lista`.



Pontificia Universidad Católica de Chile
Escuela de ingeniería
Departamento de ciencias de la
computación Profesor: Hernán Cabrera

En el DOM, dentro del elemento `<ol id="lista">`, se agrega un nuevo ítem de lista:

```
<ol id="lista">
```

```
...
```

```
<li id="nuevo">HOLA MUNDO</li>
```

```
</ol>
```

1,5 punto por identificar la creación de elementos en el DOM

1,5 punto por identificar la vinculación de los elementos ya creados en el DOM

BONUS: puede elegir un **máximo de 2** de las siguientes preguntas. Si responde más de 2 preguntas bonus se corregirán **solo las 2 primeras respuestas.**

Nota: el puntaje es un todo o nada, no se entregan puntajes parciales en estos BONUS. No se descuenta por preguntas incorrectas.

19. Explique cuándo usar OAuth, OpenID y SSO. puede ser con ejemplos (3 puntos)

OAuth: Es un protocolo para autorizar a una aplicación externa a acceder a recursos protegidos en nombre del usuario, sin compartir la contraseña del usuario con esa aplicación. Se utiliza cuando quieres permitir que una app acceda a datos privados en otro servicio, pero sin darle acceso total a la cuenta del usuario.

Ejemplo: Una app de gestión de fotos quiere acceder a tus fotos almacenadas en Google Fotos. Usas OAuth para autorizar solo ese acceso específico, sin darle tu contraseña de Google.

OpenID es un protocolo para autenticar la identidad del usuario (es decir, iniciar sesión) usando un proveedor externo confiable. Se podría utilizar cuando quieres que los usuarios puedan iniciar sesión en tu app usando una cuenta existente (Google, Facebook, etc.), sin crear una cuenta nueva. Ejemplo: Un sitio web permite que te loguees con tu cuenta de Google usando OpenID Connect para verificar quién eres.

SSO permite que un usuario inicie sesión una sola vez y tenga acceso a múltiples aplicaciones o servicios sin volver a autenticarse. Se utiliza en entornos empresariales o plataformas con varios servicios, para mejorar la experiencia del usuario evitando múltiples inicios de sesión. Ejemplo: En una empresa, al iniciar sesión en el portal corporativo, automáticamente tienes acceso al correo, calendario y sistema de gestión sin ingresar credenciales otra vez.

Esto está en ppt de las clases. 1 punto por cada explicación.



Pontificia Universidad Católica de Chile
Escuela de ingeniería
Departamento de ciencias de la
computación Profesor: Hernán Cabrera

20. Explique con sus palabras lo que significa que JavaScript implementa herencia prototipada y de un ejemplo de cómo esta implementación puede ser útil en la reutilización de código. Puede explicar con un diagrama (3 puntos)

En JavaScript, los objetos heredan directamente de otros objetos, en vez de usar clases clásicas como en otros lenguajes. Esto se llama herencia prototipada.

Cada objeto tiene un enlace interno llamado prototype que apunta a otro objeto, su prototipo. Cuando accedes a una propiedad o método de un objeto, si no lo encuentra en sí mismo, busca en su prototipo, y así sucesivamente en la cadena de prototipos. Esto permite compartir métodos y propiedades entre objetos sin necesidad de duplicar código.

A continuación un ejemplo. perro hereda el método hablar de animal si no tuviera uno propio. Esto evita repetir la función hablar para cada tipo de animal, reutilizando código

```
const animal = {  
  hablar() {  
    console.log("El animal hace un sonido");  
  }  
};  
  
const perro = Object.create(animal); // perro hereda de animal  
perro.hablar = function() {  
  console.log("Guau guau");  
};  
  
perro.hablar(); // Output: Guau guau
```

1,5 por explicar (ilustrar) herencia prototipada

1,5 por reutilización de código (por ejemplo vincular objetos funciones entre sí)

21. Explique el loop de eventos y la cola que realiza NodeJs con funciones que pueden ser bloqueantes de I/O. ¿Cómo y cuándo se vacía



Pontificia Universidad Católica de Chile
Escuela de ingeniería
Departamento de ciencias de la
computación Profesor: Hernán Cabrera

la cola? ¿Cuál es el efecto que esto tiene en la ejecución lineal de un programa de scripting como JS? ¿Qué implementó entonces JS para solucionar posibles efectos adversos? (3 puntos)

El event loop es un mecanismo que permite a Node.js manejar operaciones de entrada/salida (I/O) de forma asíncrona sin bloquear la ejecución del programa. Funciona como un ciclo que revisa continuamente una cola de tareas pendientes (callbacks, promesas resueltas, timers, etc.) y las ejecuta cuando el thread principal está libre. Gracias al event loop, Node.js puede procesar muchas operaciones concurrentes sin crear múltiples threads.

Las operaciones de I/O (como lectura de archivos, acceso a bases de datos o redes) son generalmente lentas y no deben bloquear el thread principal. Cuando una función bloqueante de I/O es llamada, Node.js la delega a un thread pool (grupo de threads) en el sistema operativo para que se ejecute en segundo plano. Una vez que la operación termina, el callback asociado se pone en la cola de tareas pendientes. El event loop vacía esta cola cuando el thread principal está libre, es decir, después de que se terminan las tareas sincronizadas y se procesa la siguiente operación. Si hay muchas tareas en la cola, la ejecución de otras operaciones se retrasa, afectando la fluidez del programa.

Para manejar mejor la asincronía y evitar el anidamiento excesivo de callbacks y mejorar la legibilidad, JavaScript implementó las promesas y más recientemente el `async/await`. Las promesas permiten encadenar operaciones asíncronas de forma clara y controlada. `async/await` es syntactic sugar que hace que el código asíncrono se escriba de manera similar al código síncrono, facilitando la comprensión y manejo de errores. Estas herramientas permiten escribir código que espera a que se resuelvan operaciones asíncronas sin bloquear el event loop ni saturar la cola.

1 punto por explicar el loop de eventos y su función

1 punto por explicar el impacto en asincronía (con el pool de funciones bloqueantes I/O) y 1 punto por implementación de promesas o de `async/await`

22. Con respecto a PUT, PATCH, POST. ¿cual de ellos es idempotente? ¿Qué significa idempotencia? de un ejemplo. (3 puntos)

PUT es un método idempotente.

PATCH y POST no son idempotentes.



Pontificia Universidad Católica de Chile
Escuela de ingeniería
Departamento de ciencias de la
computación Profesor: Hernán Cabrera

Un método HTTP es idempotente si realizar la misma petición varias veces produce el mismo resultado que hacerla una sola vez. Es decir, no importa si envías la misma solicitud repetidas veces, el estado del recurso en el servidor no cambiará más allá del primer cambio.

Ejemplo: Supongamos que quieres actualizar un recurso (por ejemplo, un usuario) en la URL `/usuarios/123` con los datos:

```
{  
  "nombre": "Valeria",  
  "email": "valeria@example.com"  
}
```

Si haces la petición PUT una vez o diez veces seguidas con los mismos datos, el resultado en el servidor será el mismo: el usuario 123 tendrá esos datos actualizados.

No se crearán recursos duplicados ni se modificarán múltiples veces.

1 por explicar que PUT es idempotente

2 por explicar idempotencia

23. En Koa existe `ctx` y `next`. ¿Para qué sirve cada uno de ellos? de un ejemplo sencillo de uso de `next` y explique como sería la ejecución de su código ejemplo A modo de recuerdo, este trozo de código `app.use(async ( ctx, next)=>(...codigo...))` (2 puntos)

`ctx` es el contexto de la solicitud HTTP en Koa. Este contiene toda la información sobre la petición (`ctx.request`), la respuesta (`ctx.response`), encabezados, estado, etc. Es el objeto principal que manejas para leer y escribir datos en la app.

`next` es una función que representa el siguiente middleware en la cadena. Cuando llamas a `await next()`, le dices a Koa que pase el control al siguiente middleware y espere a que termine antes de continuar la ejecución. Esto permite la anidación y el control del flujo de middlewares.

Ejemplo:

```
app.use(async (ctx, next) => {  
  console.log("Middleware 1: Antes de next");  
  await next(); // Pasa al siguiente middleware  
  console.log("Middleware 1: Después de next");  
});
```

```
app.use(async (ctx, next) => {
```



Pontificia Universidad Católica de Chile
Escuela de ingeniería
Departamento de ciencias de la
computación Profesor: Hernán Cabrera

```
console.log("Middleware 2: Antes de next");
ctx.body = "¡Hola desde Koa!";
await next();
console.log("Middleware 2: Después de next");
});
```

1 por explicar ctx (contexto) y next (anidación en la ejecución en Koa)
1 por un ejemplo (no tiene que ser totalmente funcional, es importante que muestre anidación)

24. Para el siguiente código (4 puntos) explique por qué la salida en pantalla (los console.log del final) es undefined true true

1) **console.log(obj.valor); → undefined**

obj es una instancia de Hn.

El constructor Hn define solo this.nombre = "H", no define valor.

valor está definido en el constructor Fn como this.valor = 1;.

Pero en ningún punto se llama a Fn para inicializar valor dentro de la cadena de constructores.

Por eso, al acceder obj.valor, no encuentra esa propiedad en obj ni en su prototipo (porque es una propiedad de instancia, no prototipo), entonces devuelve undefined.

2) **console.log('sumar' in obj); → true**

'sumar' es un método definido en Fn.prototype.

El prototipo de obj (que es Hn.prototype) hereda de Gn.prototype, que a su vez hereda de Fn.prototype.

Por la cadena de prototipos, obj tiene acceso a la propiedad sumar.

Por eso 'sumar' in obj es true.

3) **console.log(obj instanceof Fn); → true**

El operador instanceof revisa si el prototipo de Fn está en la cadena de prototipos de obj.

Como Hn.prototype hereda de Gn.prototype y Gn.prototype hereda de Fn.prototype, Fn.prototype está en la cadena de prototipos de obj.

Por lo tanto, obj instanceof Fn es true

1.35 Explicar undefined: No se inicializó valor en el constructor

1.35 Explicar true: sumar está en la cadena de prototipo

1.3 Explicar true: Prototipo de Fn está en la cadena prototípica



Pontificia Universidad Católica de Chile
Escuela de ingeniería
Departamento de ciencias de la
computación Profesor: Hernán Cabrera